

Q2. What is Integrity Constraint. Explain different key attribute.?

Integrity constraints are rules that help to maintain the accuracy and consistency of data in a database. They can be used to enforce business rules or to ensure that data is entered correctly.

1. Primary Key Constraints

A primary key constraint (also known as a "primary key") is a type of key constraint that requires every value in a given column to be unique. In other words, no two rows in a table can have the same value for their primary key column(s).

```
Sql> CREATE TABLE Student(  
    Std_Enroll_No int(10) PRIMARY KEY,  
    Std_Name char(20),  
    Class int(5),  
    Address varchar(20)  
);
```

2. Unique Key Constraints

A unique key constraint is a column or set of columns that ensures that the values stored in the column are unique. A table can have more than one unique key constraint, unlike the primary key. A unique key column can contain NULL values. Like primary keys, unique keys can be made up of a single column or multiple columns.

```
Sql> CREATE TABLE Student(  
    Std_Enroll_No int(10) UNIQUE,  
    Std_Name char(20),  
    Class int(5),  
    Address varchar(20)  
);
```

3. Foreign Key Constraints

A foreign key constraint defines a relationship between two tables. A foreign key in one table references a primary key in another table. Foreign keys prevent invalid data from being inserted into the foreign key column. Foreign keys can reference a single column or multiple columns.

```
Sql> CREATE TABLE Student(  
    Std_Enroll_No int(10) PRIMARY KEY,  
    Std_Name char(20),  
    Class int(5),  
    Address varchar(20)  
);
```

```

Sql> CREATE TABLE Faculty(
    F_Id int(10) PRIMARY KEY
    Std_Enroll_No int(10) REFERENCES Student,
    F_Name char(20) Class int(5),
    Address varchar(20)
);
Or
Sql> CREATE TABLE Faculty(
    F_Id int(10) PRIMARY KEY
    F_Name char(20) Class int(5),
    Address varchar(20),
    FOREIGN KEY (Std_Enroll_No) REFERENCES Students(Std_Enroll_No)
);

```

4. NOT NULL Constraints

A NOT NULL constraint is used to ensure that no row can be inserted into the table without a value being specified for the column(s) with this type of constraint. Thus, every row must have a non-NULL value for these columns.

```

Sql> CREATE TABLE Student(
    Std_Enroll_No int(10),
    Std_Name char(20) NOT NULL,
    Class int(5),
    Address varchar(20)
);

```

5. Check Constraints

A check constraint enforces data integrity by allowing you to specify conditions that must be met for data to be inserted into a column. For example, you could use a check constraint to ensure that only positive integer values are inserted into a particular column. Check constraints are usually used in combination with other constraints (such as NOT NULL constraints) to enforce more complex rules.

```

Sql> CREATE TABLE Student(
    Std_Enroll_No int(10),
    Std_Name char(20),
    Age int(5) CHECK(Age>=21);
    Address varchar(20)
);

```

6. Default Constraint: A Default constraint is used to store 'default Value' if user does not filled or store any value.

```
Sql> CREATE TABLE Student(  
    Std_Enroll_No int(10),  
    Std_Name char(20) NOT NULL,  
    Class int(5),  
    Address varchar(20) DEFAULT 'Indore';  
);
```